

PES UNIVERSITY, Bangalore
Established under Karnataka Act No. 16 of 2013
Department of Computer Science & Engineering
UE24CS341A : Software Engineering

MINI PROJECT

Name	SRN	Section: F
Kari Bhavitha	PES1UG25CS821	
Pavana P	PES1UG24CS320	
Prarthana Shivakumar	PES1UG24CS339	
Raghavendrah	PES1UG24CS356	
Faculty guide	Ashok Kumar Patil	

15. Simple Shell (Command Line Interpreter)

Software Test Plan (STP)

Table of Contents

1. Introduction	2
2. Test Items	2
3. Features to be Tested.....	3
4. Features not to be Tested.....	6
5. Test Approach / Strategy	7
5.1 Security Validation	8
6. Test Environment.....	8
7. Test Schedule.....	9
8. Test Deliverables	9
9. Roles and Responsibilities.....	9
10. Risks and Mitigation	9
11. Assumptions & Dependencies	10
12. Suspension & Resumption Criteria.....	10
13. Test Case Management & Traceability.....	10
14. Test Metrics & Reporting.....	12
15. Approvals	13
Appendix A - Test Case Specifications	13

Appendix B - Test Case Catalogue and Results.....	17
B.1 Unit test cases.....	17
B.2 Functional test cases.....	19
B.3 Integration test cases.....	20
B.4 Error / negative test cases	21
B.5 Robustness test cases.....	22
B.6 Security test cases.....	23
B.7 Non-functional test cases.....	24
B.8 System test cases	25
B.9 Tests still to be executed (not run, no result claimed)	26

Status: Draft 1.0 - Part-1 submission (test cases designed and executed once; results in section 14 and Appendix B)

1. Introduction

Purpose: This document defines the test plan for mysh v1.0: objectives, scope, strategy, environment, schedule, responsibilities and the test cases that verify the functional, non-functional and security requirements of the SRS.

Scope: Testing covers command execution, parsing, built-ins, environment and expansion, redirection, pipelines, sequencing, background jobs, signal handling, termination, the non-functional requirements (performance, robustness, usability, maintainability, portability, memory safety, reliability) and the security requirements SR-01 to SR-10. Behaviour of the operating system, the terminal emulator and external utilities is excluded.

References: SRS v1.0 (docs/SRS.docx); Software Architecture and Design Specification v1.0 (docs/SAD.docx); RTM (SRS Appendix C); IEEE Std 829-2008 (software test documentation); POSIX.1-2008 Shell Command Language; the course template Test_Plan_Template_for_SE.docx.

Definitions: STP (Software Test Plan), SRS (Software Requirements Specification), RTM (Requirement Traceability Matrix), FR (functional requirement), NFR (non-functional requirement), SR (security requirement), pty (pseudo-terminal), ASan/UBSan/LSan (Address/Undefined-behaviour/Leak sanitizers), fuzzing (testing with large numbers of random inputs).

2. Test Items

Test item	Source	Requirements
Shell Core: prompt, read-eval loop, signals, options, termination	main.c, shell.c	FR-01, FR-02, FR-19, FR-21, FR-22, SR-04, SR-08, SR-09
Parser: tokenizer, quoting, expansion, syntax errors	parser.c	FR-03, FR-04, FR-13, SR-01, SR-02, SR-06
Executor: fork/exec, wait, redirection, pipes, background jobs	executor.c	FR-05, FR-06, FR-07, FR-14 to FR-18, FR-20, SR-03, SR-05, SR-08, SR-10
Built-in commands: cd, pwd, exit,	builtins.c	FR-08 to FR-12, SR-07

Test item	Source	Requirements
help, export, unset, env		
Utilities	utils.c	NFR-08, SR-06, SR-07
Build system and test tooling	Makefile, tests/	NFR-04, NFR-05, NFR-07

3. Features to be Tested

Features are mapped to SRS requirement IDs. Pass criteria are in the SRS (section 5 and 6) and in the expected results of the test cases.

Requirement ID	Feature	Type	Priority	Test cases
FR-01	Interactive prompt and read-eval loop	Functional	High	ST-01, ST-02, ST-10, TC-47, UT-18
FR-02	Non-interactive input sources	Functional	High	ST-10, TC-50, TC-51, TC-62, TC-63, TC-76
FR-03	Tokenization and quoting	Functional	High	ST-10, TC-37, TC-39, TC-40, UT-01, UT-02, UT-05, UT-06
FR-04	Syntax error detection and reporting	Functional	High	ST-10, TC-41, TC-42, TC-43, TC-44, TC-45, TC-46, TC-70, TC-73, UT-12, UT-13, UT-14, UT-15, UT-16, UT-22, UT-23
FR-05	External command execution with PATH search	Functional	High	ST-02, ST-10, TC-02, TC-03, TC-10, TC-48, TC-49
FR-06	Foreground wait and exit status	Functional	High	ST-04, ST-10, TC-30, TC-56
FR-07	Invalid command handling	Functional	High	ST-10, TC-08, TC-09, TC-66
FR-08	Built-in cd	Functional	High	ST-10, TC-04, TC-05, TC-06, TC-07, TC-65
FR-09	Built-in pwd	Functional	High	ST-10, TC-01
FR-10	Built-in exit	Functional	High	ST-09, ST-10, TC-31, TC-32, TC-33, TC-53
FR-11	Built-in help	Functional	Medium	ST-10, TC-34
FR-12	Environment variables	Functional	High	ST-10, TC-23, TC-24, TC-25, TC-26, TC-27, TC-64, UT-19
FR-13	Variable and tilde expansion	Functional	High	ST-10, TC-28, TC-29, TC-38, TC-60, TC-61, TC-72, UT-03, UT-04, UT-17, UT-20, UT-21
FR-14	Output redirection >	Functional	High	ST-10, TC-12, TC-13, TC-22, TC-71, UT-08, UT-24
FR-15	Append redirection >>	Functional	High	ST-10, TC-14, TC-15, UT-09
FR-16	Input redirection <	Functional	High	ST-10, TC-11

Requirement ID	Feature	Type	Priority	Test cases
FR-17	Redirection error handling	Functional	High	ST-10, TC-16, TC-17
FR-18	Pipelines	Functional	High	ST-10, TC-18, TC-19, TC-20, TC-21, TC-57, TC-58, UT-07
FR-19	Command sequencing with ;	Functional	Medium	ST-10, TC-35, UT-11
FR-20	Background execution with &	Functional	High	ST-05, ST-07, ST-10, ST-11, TC-36, TC-74, TC-75, UT-10
FR-21	Signal handling	Functional	High	ST-03, ST-04, ST-07, ST-08, ST-10, ST-11
FR-22	Graceful termination	Functional	High	ST-06, ST-10, TC-52

Requirement ID	Feature (measurable criterion in SRS 6.4)	Type	Test cases
NFR-01	Performance - When a pipeline is terminated by &, the shell shall return to the prompt within 500 ms of reading the line, regardless of how long the job runs.	Non-functional	NF-01, NF-02, NF-03
NFR-02	Robustness - The shell shall process without crash, hang or memory error: a 100000-character argument, a command with 2000 arguments, a 10-stage pipeline, 300 consecutive pipelines, and 200000 randomly generated command lines in the parser fuzz test (0 failures allowed).	Non-functional	SC-12, SC-13, ST-12, TC-54, TC-55, TC-57, TC-77
NFR-03	Usability - 100% of diagnostics shall be written to stderr as 'mysh: ' (usually 'mysh: '); help shall list all 7 built-ins in at most 25 lines of at most 80 characters; the interactive prompt shall show the current working directory.	Non-functional	NF-04, NF-08, ST-01, TC-34
NFR-04	Maintainability - All source files shall compile with 0 warnings under gcc -std=c11 -Wall -Wextra -Wpedantic -Werror; each of the 6 modules shall publish its interface in a header file; adding a built-in shall require only one new entry in the built-in table and one new function.	Non-functional	NF-05
NFR-05	Portability - The sources shall compile with 0 warnings both as C11 and as C99 with POSIX.	Non-functional	NF-06
NFR-06	Security - The security requirements SR-01 to SR-10 of section 6.	Non-functional	SC-01, SC-02, SC-03, SC-04, SC-05, SC-06, SC-07, SC-08, SC-09,

Requirement ID	Feature (measurable criterion in SRS 6.4)	Type	Test cases
			SC-10, SC-11, SC-12, SC-13, ST-03, ST-04, ST-08, ST-12
NFR-07	Testability - Every functional requirement shall have at least one automated test that runs without a terminal (-c, script file or stdin), and the whole suite shall be runnable with a single command (make test) that returns a non-zero status on any failure.	Non-functional	TC-50, TC-51, TC-62
NFR-08	Memory safety and resource management - The unit and functional suites and the fuzz run shall produce 0 reports under AddressSanitizer, UndefinedBehaviorSanitizer and LeakSanitizer, and after 300 consecutive pipelines an executed program shall see no open descriptor other than 0, 1 and 2.	Non-functional	SC-09, ST-12, TC-77
NFR-09	Reliability - After each of 10 different kinds of user error (unknown command, bad cd, missing redirection file, syntax error, bad export, non-executable file, unsupported operator, .	Non-functional	NF-07, ST-03, TC-05, TC-08, TC-16, TC-17, TC-41

Requirement ID	Security feature (objective)	Type	Priority	Test cases
SR-01	The shell shall pass every word to an executed program only as one element of the argv vector given to execvp(). (SO-1)	Security	High	SC-01, SC-03, SC-04
SR-02	Text produced by expansion (\$NAME, \${NAME}, \$?, \$\$, ~) shall not be scanned again for operators, quotes or further expansions: a variable whose value is 'a; echo pwned' shall be passed as the single argument 'a; echo pwned'. (SO-1)	Security	High	SC-02
SR-03	The shell shall not call setuid, setgid, seteuid, setegid, setreuid or setresuid, and every executed program shall run with the same user and group ids as the shell. (SO-2)	Security	High	SC-05
SR-04	The shell shall not create any file on its own initiative (no history, log or state file); the only files it creates are those named by the user in > or >> redirections. (SO-3)	Security	Medium	SC-06
SR-05	Files created by > or >> shall be created with mode 0644 modified by the process umask, so the resulting	Security	Medium	SC-07

Requirement ID	Security feature (objective)	Type	Priority	Test cases
	permissions are never more permissive than 0644 (for example 0600 under umask 077). (SO-3)			
SR-06	The parser and executor shall handle every input line without memory errors and without fixed-size buffers for user input; a fuzz run of 200000 random inputs in the parser and 600 random command lines in the real shell shall give 0 crashes, hangs or sanitizer reports. (SO-4)	Security	High	SC-12, SC-13, ST-12
SR-07	Environment variable names accepted by export, unset and NAME=VALUE shall match [A-Za-z_][A-Za-z0-9_]*; any other name shall be rejected with status 1 and an error message, leaving the environment unchanged. (SO-4)	Security	Medium	SC-08
SR-08	Executed programs shall inherit no open file descriptor other than 0, 1 and 2: pipe ends and redirection descriptors shall be closed in every child and the script file descriptor shall be close-on-exec. (SO-5)	Security	High	SC-09, SC-10
SR-09	The shell shall keep running when SIGINT or SIGTSTP (Ctrl+C, Ctrl+Z) is delivered at the prompt or during a foreground job, and the SIGINT handler shall only set a flag (async-signal-safe). (SO-5)	Security	High	ST-03, ST-04, ST-08
SR-10	A background job shall not read from the shell's standard input unless the user redirects it: its standard input shall be /dev/null. (SO-5)	Security	Medium	SC-11

4. Features not to be Tested

- Linux kernel, glibc and the terminal emulator (assumed correct; vendor responsibility).
- Behaviour of external programs the shell launches (ls, grep, sort, sleep, ...): only that the shell starts them correctly and reports their status.
- Features outside the product scope (SRS 2.5): job control (fg, bg, jobs), wildcard expansion, command substitution, &&, ||, 2>, here-documents, aliases, functions, history, line editing. Only the *rejection* of && and || is tested (TC-45, TC-73).
- Operating systems other than Ubuntu 24.04 (portability beyond compilation as C99/C11 is not tested).
- Full-screen terminal programs (vi, less) and long-duration soak tests.
- Third-party penetration testing and formal certification.

5. Test Approach / Strategy

Levels

Level	What is tested	Technique / tool	Test IDs
Unit	Parser and utility functions in isolation (no processes)	C driver linked with the modules (tests/test_parser.c)	UT-01 to UT-24
Integration	Parser + executor + built-ins + OS together: pipes with redirection, built-ins in pipelines, environment inheritance, script/stdin input	bash harness tests/run_tests.sh using mysh -c, script file and stdin	TC-18 to TC-22, TC-24, TC-48 to TC-53, ...
System	End-to-end behaviour on a real terminal: prompt, Ctrl+C, Ctrl+D, Ctrl+Z, job notices; demo script; sanitizer build	Python pty driver tests/interactive_test.py, demo/demo.mysh, make sanitize	ST-01 to ST-12
Acceptance	Demonstration to the instructor following docs/DEMO_SCRIPT.md	Manual	Planned (after Part-1)

Types

- *Functional testing*: each FR, black-box, exact stdout and exit status (TC-xx).
- *Negative / error testing*: invalid commands, syntax errors, missing files, bad arguments (TC-05, TC-08, TC-09, TC-16, TC-17, TC-33, TC-41 to TC-46, ...).
- *Regression testing*: the whole suite is re-run with one command (make test) after every change.
- *Performance testing*: response time of background start and throughput of 1000 external and 1000 built-in commands (NF-01 to NF-03). Thresholds are deliberately generous so that they detect regressions, not hardware differences.
- *Robustness testing*: 100,000-character argument, 2,000 arguments, 10-stage pipeline, 300 consecutive pipelines (TC-54, TC-55, TC-57, TC-77).
- *Usability testing (objective part)*: diagnostic format and help-screen size (NF-04, NF-08); subjective usability by demonstration.
- *Maintainability/portability checks*: compilation with -Werror as C11 and as C99 (NF-05, NF-06).
- *Security testing*: section 5.1.

Entry criteria: the build succeeds without warnings; the test environment of section 6 is available; fixtures are generated by the harness itself.

Exit criteria: 100% of planned test cases executed; 0 failing; 0 open critical or high defects; every FR, NFR and SR traced to at least one passing test (section 13); sanitizer and fuzz runs report nothing.
Status for this release: all criteria are met (section 14).

Pass/fail rule: a test passes only if stdout, exit status and (where specified) stderr text all match the expected result exactly; timing tests pass if the measured time is within the stated bound.

Defect severity: Critical (crash, data loss, security breach), High (requirement not met), Medium (wrong message or status), Low (cosmetic or test-only).

5.1 Security Validation

Validation targets the security objectives SO-1 to SO-5 and requirements SR-01 to SR-10 (SRS 6.3) and the STRIDE threats of the SAD (section 3.9).

Activity	What it validates	Requirement	Test IDs	Status
Static check: no system(), popen() or sh -c in the source	No second interpretation of user text	SR-01	SC-04	PASS
Injection tests: quoted metacharacters, variable contents holding operators, file names containing ; \	Quoting and expansion never create syntax	SR-01, SR-02	SC-01, SC-02, SC-03	PASS
Privilege check: no setuid-family calls; child uid equals shell uid	No privilege change	SR-03	SC-05	PASS
Data exposure: no history/log files; file modes under umask 077/022/000; background job stdin	No unintended disclosure	SR-04, SR-05, SR-10	SC-06, SC-07, SC-11	PASS
Environment validation: invalid names rejected	Environment integrity	SR-07	SC-08, TC-64	PASS
Descriptor inheritance after 300 pipelines and in script mode	No leaked descriptors	SR-08	SC-09, SC-10	PASS
Signal robustness: Ctrl+C at prompt and during a job, Ctrl+Z	Shell cannot be killed/frozen by keyboard signals	SR-09	ST-03, ST-04, ST-08	PASS
Fuzzing, parser level: 200,000 pseudo-random lines (metacharacters, quotes, expansions, raw bytes) under ASan+UBSan with structural invariants	Memory safety of the parser	SR-06	SC-12	PASS
Fuzzing, process level: 600 random command lines run through the sanitizer build of the real shell	No crash/hang/sanitizer report end to end	SR-06	SC-13	PASS
Sanitizer build of unit + functional suites (ASan, UBSan, LSan)	Memory errors and leaks	SR-06, NFR-08	ST-12	PASS

Planned but not performed: independent penetration test by a third party; seccomp/sandbox review; fuzzing with a coverage-guided fuzzer (AFL/libFuzzer); static analysis with a dedicated tool (cppcheck/clang-analyzer were not available in the test environment).

6. Test Environment

- **Hardware:** x86-64 virtual machine, Linux 6.18 kernel.
- **Software:** Ubuntu 24.04.4 LTS, gcc 13.3.0, GNU make, bash, Python 3 (pty), coreutils, grep, sort, sleep, id, stat.
- **Tools:** bash harness (functional), Python pty driver (system), C unit driver, AddressSanitizer/UBSan/LeakSanitizer, own fuzzers (tests/fuzz_parser.c, tests/fuzz_shell.py), git-style folders for result files tests/results_.*.

- **Test data:** generated per run in a temporary directory: file.txt (3 lines), input.txt, a non-executable noexec.sh, an executable bin/hello_tool, scripts for stdin/script modes; no real user data.
- **Defect tracking:** table in section 14 (a tracker such as Jira is not used for this project).

7. Test Schedule

Milestone	Date	Status
Test case design (unit, functional, system) and automation	up to 2026-10-03	Done
Security and non-functional test design (SC, NF), fuzzers	2026-10-03	Done
Environment setup	2026-10-03	Done
Test execution round 1 (all 130 tests)	2026-10-03	Done: 130 passed, 0 failed
Regression run after review comments	05-Oct-2026	Planned
Part-1 submission	07-Oct-2026	Planned
Acceptance demonstration	As scheduled by the course after Part-1	Planned

8. Test Deliverables

- Test Plan (this document)
- Test cases: manual specifications (Appendix A) and automated tests (tests/test_parser.c, tests/run_tests.sh, tests/interactive_test.py, tests/fuzz_parser.c, tests/fuzz_shell.py)
- Test scripts and test data generators (the same files)
- Test execution logs: tests/results_unit.txt, results_functional.txt/.tsv, results_interactive.txt, results_fuzz.txt, results_sanitizer.txt
- Defect report (section 14)
- Test Summary Report (section 14)

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Member 4	Prepare plan, own harness and traceability, coordinate execution, sanitizer and fuzz runs
Test Engineer	Member 2	Parser unit tests, expansion/built-in tests, parser fuzzer
Test Engineer	Member 3	Redirection, pipe, background and signal tests (pty), security tests SC-xx
Developer	Member 1	Support defect fixes and triage, core and executor changes
Product Owner / reviewer	Faculty guide	Approve test results, sign off readiness

10. Risks and Mitigation

Risk	Mitigation
Timing tests fail on a slow or busy machine	Thresholds are generous (500 ms for a 2 s job; 5 s for 1000 commands); measured values are recorded (4 ms and about 0.8 s here)

Risk	Mitigation
Pseudo-terminal tests are timing sensitive	Generous waits and timeouts; each test starts its own shell; failures print captured output
Background children keep a captured pipe open and make the <i>test harness</i> wait	Harness writes background-test output to files (seen once: TC-36/TC-74, section 14)
Sanitizer or tool not available on a team member's machine	Same suite runs without sanitizers (make test); sanitizer run done on the reference environment
Tests run as root hide permission problems	Permission tests use mode bits and the exec bit (TC-09 passes as root); re-run as an ordinary user is listed as a regression step
Only one platform available	Stated limitation; portability test limited to compilation as C99/C11

11. Assumptions & Dependencies

- The reference environment of section 6 stays available and PATH contains the standard utilities.
- Tests create and delete their own temporary directories; no network is needed.
- The terminal emulation of Python's pty module delivers Ctrl+C as SIGINT to the foreground process group.
- gcc supports -fsanitize=address,undefined.

12. Suspension & Resumption Criteria

Suspend testing if: the build fails to compile; the environment is unavailable for more than 4 hours; the build is so unstable that more than 30% of test cases are blocked. Resume when: the blocking defects are fixed and a clean build passes the smoke subset (UT-01 to UT-10, TC-01 to TC-03).

13. Test Case Management & Traceability

The RTM maps every SRS requirement to its architecture/design element, source file and test cases (SRS Appendix C). The test side of that matrix is reproduced here and is generated from the same data as the result files, so IDs cannot drift. Test IDs: UT unit, TC functional/integration/negative/robustness, SC security, NF non-functional, ST system.

Coverage: functional requirements with at least one test: 22 of 22; security requirements: 10 of 10; non-functional requirements: 9 of 9.

Requirement	Unit / functional / security / non-functional tests	System tests
FR-01 Interactive prompt and read-eval loop	TC-47, UT-18	ST-01, ST-02, ST-10
FR-02 Non-interactive input sources	TC-50, TC-51, TC-62, TC-63, TC-76	ST-10
FR-03 Tokenization and quoting	TC-37, TC-39, TC-40, UT-01, UT-02, UT-05, UT-06	ST-10
FR-04 Syntax error detection and reporting	TC-41, TC-42, TC-43, TC-44, TC-45, TC-46, TC-70, TC-73, UT-12, UT-13, UT-14, UT-15,	ST-10

Requirement	Unit / functional / security / non-functional tests	System tests
	UT-16, UT-22, UT-23	
FR-05 External command execution with PATH search	TC-02, TC-03, TC-10, TC-48, TC-49	ST-02, ST-10
FR-06 Foreground wait and exit status	TC-30, TC-56	ST-04, ST-10
FR-07 Invalid command handling	TC-08, TC-09, TC-66	ST-10
FR-08 Built-in cd	TC-04, TC-05, TC-06, TC-07, TC-65	ST-10
FR-09 Built-in pwd	TC-01	ST-10
FR-10 Built-in exit	TC-31, TC-32, TC-33, TC-53	ST-09, ST-10
FR-11 Built-in help	TC-34	ST-10
FR-12 Environment variables	TC-23, TC-24, TC-25, TC-26, TC-27, TC-64, UT-19	ST-10
FR-13 Variable and tilde expansion	TC-28, TC-29, TC-38, TC-60, TC-61, TC-72, UT-03, UT-04, UT-17, UT-20, UT-21	ST-10
FR-14 Output redirection >	TC-12, TC-13, TC-22, TC-71, UT-08, UT-24	ST-10
FR-15 Append redirection >>	TC-14, TC-15, UT-09	ST-10
FR-16 Input redirection <	TC-11	ST-10
FR-17 Redirection error handling	TC-16, TC-17	ST-10
FR-18 Pipelines	TC-18, TC-19, TC-20, TC-21, TC-57, TC-58, UT-07	ST-10
FR-19 Command sequencing with ;	TC-35, UT-11	ST-10
FR-20 Background execution with &	TC-36, TC-74, TC-75, UT-10	ST-05, ST-07, ST-10, ST-11
FR-21 Signal handling	-	ST-03, ST-04, ST-07, ST-08, ST-10, ST-11
FR-22 Graceful termination	TC-52	ST-06, ST-10

Requirement	Tests (including inspection where stated)
NFR-01 Performance	NF-01, NF-02, NF-03
NFR-02 Robustness	TC-54, TC-55, TC-57, TC-77, SC-12, SC-13
NFR-03 Usability	NF-04, NF-08, TC-34, ST-01
NFR-04 Maintainability	NF-05 (compiler); inspection of builtins.c table
NFR-05 Portability	NF-06, whole suite on Ubuntu 24.04
NFR-06 Security	SC-01 to SC-13, ST-03, ST-04, ST-08, ST-12
NFR-07 Testability	RTM (Appendix C); make test
NFR-08 Memory safety and resource management	ST-12, SC-09, TC-77

Requirement	Tests (including inspection where stated)
NFR-09 Reliability	NF-07, TC-05, TC-08, TC-16, TC-41, ST-03

Requirement	Objective	Tests
SR-01	SO-1	SC-01, SC-03, SC-04
SR-02	SO-1	SC-02
SR-03	SO-2	SC-05
SR-04	SO-3	SC-06
SR-05	SO-3	SC-07
SR-06	SO-4	SC-12, SC-13, ST-12
SR-07	SO-4	SC-08
SR-08	SO-5	SC-09, SC-10
SR-09	SO-5	ST-03, ST-04, ST-08
SR-10	SO-5	SC-11

14. Test Metrics & Reporting

Test Summary Report (execution of 2026-10-03 on Ubuntu 24.04.4, gcc 13.3.0)

Category	Designed	Executed	Passed	Failed
Unit (UT)	24	24	24	0
Functional / integration / negative / robustness (TC)	73	73	73	0
Security (SC)	13	13	13	0
Non-functional (NF)	8	8	8	0
System (ST)	12	12	12	0
Total	130	130	130	0

Metrics

Metric	Value
% of test cases executed	100% (130 of 130)
% passed / failed	100.0% / 0.0%
Requirement coverage (FR / NFR / SR with at least one test)	22/22, 9/9, 10/10 (100%)
Product defects found during testing	1 (D-3), fixed and verified
Defect density (product)	0.70 defects per KLOC (1433 lines of C in src/)
Defect aging	All defects found and closed within the same test cycle (open: 0)
Open critical/high defects	0

Defect log

ID	Where	Description	Severity	Resolution
D-	Test	TC-36 and TC-74 first failed: the harness	Low	Tests changed to

ID	Where	Description	Severity	Resolution
1	harness	captured output with \$(...), which stays open while a background sleep holds the pipe, so the <i>harness</i> waited 2 s. The shell was correct.	(test)	write to a file; shell unchanged
D-2	Makefile / tests	make sanitize ran the bash-syntax harness with sh (dash) and gave 10 false failures.	Low (test)	Makefile now calls bash
D-3	Product (security, SR-08)	The script file descriptor was inherited by every program started from a script (mysh script followed by a program listing its descriptors showed fd 3). Found while designing SC-10; the new test fails against the old code (output [3]).	Medium	Script opened with close-on-exec (fopen mode re); SC-10 now passes
D-4	Test	SC-08 expected exit status 0, but the last command (grep -c) correctly returns 1 when it finds no match.	Low (test)	Expected status corrected to 1; shell unchanged

Reports: daily execution status = output of make test; final Test Summary Report = this section plus the result files in tests/.

15. Approvals

Role	Name	Signature / Date
QA Lead	Member 4	
Dev Lead	Member 1	
Product Owner / Faculty guide		

Appendix A - Test Case Specifications

19 key test cases are specified in full here (functional, negative, integration, system, security, performance, usability and reliability). The remaining automated tests are listed in Appendix B with their commands and expected results.

TC-03 - Run an external command

Field	Content
Requirement(s)	FR-05
Preconditions	mysh built; any directory
Steps / input	1. mysh -c 'echo Hello'
Expected result	stdout is exactly Hello; exit status 0
Result of execution on 2026-10-03	PASS (as expected)

TC-08 - Invalid command handling

Field	Content
Requirement(s)	FR-07
Preconditions	mysh built; no program named invalidcommand in PATH
Steps / input	1. mysh -c 'invalidcommand; echo \$?'
Expected result	stderr contains command not found; stdout is 127; shell continues

Field	Content
Result of execution on 2026-10-03	PASS (as expected)

TC-04 - cd changes the shell's own directory

Field	Content
Requirement(s)	FR-08
Preconditions	directory sub exists in the working directory
Steps / input	1. cd sub; pwd; cd ..; pwd
Expected result	first pwd prints <dir>/sub, second prints <dir>
Result of execution on 2026-10-03	PASS (as expected)

TC-12 - Output redirection >

Field	Content
Requirement(s)	FR-14
Preconditions	empty working directory
Steps / input	1. echo hello > output.txt; cat output.txt
Expected result	file output.txt is created and cat prints hello
Result of execution on 2026-10-03	PASS (as expected)

TC-18 - Pipe between two commands

Field	Content
Requirement(s)	FR-18
Preconditions	file.txt contains hello world, foo, hello again
Steps / input	1. cat file.txt \ grep hello
Expected result	stdout is the two lines containing hello; status 0
Result of execution on 2026-10-03	PASS (as expected)

TC-20 - Pipeline combined with < and > (integration)

Field	Content
Requirement(s)	FR-18
Preconditions	file.txt as above
Steps / input	1. sort < file.txt \ head -n 1 > first.txt; cat first.txt
Expected result	first.txt contains foo (alphabetically first line)
Result of execution on 2026-10-03	PASS (as expected)

TC-32 - exit stops the shell and sets the status

Field	Content
Requirement(s)	FR-10
Preconditions	none
Steps / input	1. mysh -c 'echo a; exit 3; echo b'
Expected result	stdout a only; process exit status 3

Field	Content
Result of execution on 2026-10-03	PASS (as expected)

TC-41 - Syntax error: unterminated quote

Field	Content
Requirement(s)	FR-04
Preconditions	none
Steps / input	1. mysh -c 'echo "abc'
Expected result	no command runs; stderr unterminated double quote; exit status 2
Result of execution on 2026-10-03	PASS (as expected)

TC-36 - Background execution does not block

Field	Content
Requirement(s)	FR-20
Preconditions	none
Steps / input	1. mysh -c 'sleep 2 & echo started' and measure elapsed time
Expected result	started printed; elapsed time below 1.5 s
Result of execution on 2026-10-03	PASS (as expected)

ST-04 - Ctrl+C stops a foreground job but not the shell (system, pseudo-terminal)

Field	Content
Requirement(s)	FR-21,FR-06,SR-09
Preconditions	mysh started on a pty and prompt visible
Steps / input	1. type sleep 30; 2. after 0.6 s send Ctrl+C; 3. type echo \$?
Expected result	prompt returns within 5 s; echo \$? prints 130; shell still responds
Result of execution on 2026-10-03	PASS (as expected)

ST-10 - Full demonstration script (system)

Field	Content
Requirement(s)	FR-01..FR-22
Preconditions	demo/demo.mysh present
Steps / input	1. mysh demo.mysh
Expected result	all documented outputs present (Hello, Shell!; Welcome Student; both lines of notes.txt; command not found; status was 127); exit status 0
Result of execution on 2026-10-03	PASS (as expected)

SC-01 - Injection through quoted metacharacters (security, SR-01)

Field	Content
Requirement(s)	SR-01
Preconditions	empty working directory

Field	Content
Steps / input	1. echo "a; echo injected \ cat > pwned"; ls pwned
Expected result	the whole text is printed literally; no file pwned exists (ls fails with status 2)
Result of execution on 2026-10-03	PASS (as expected)

SC-02 - No re-interpretation of variable contents (security, SR-02)

Field	Content
Requirement(s)	SR-02
Preconditions	none
Steps / input	1. X="a; echo pwned"; echo \$X
Expected result	stdout is the single line a; echo pwned
Result of execution on 2026-10-03	PASS (as expected)

SC-07 - File creation permissions (security, SR-05)

Field	Content
Requirement(s)	SR-05
Preconditions	umask set to 077, 022 and 000 in three runs
Steps / input	1. echo x > f under each umask; 2. read file mode
Expected result	modes 600, 644, 644 (never more permissive than 0644)
Result of execution on 2026-10-03	PASS (as expected)

SC-09 - No descriptor inheritance (security, SR-08)

Field	Content
Requirement(s)	SR-08
Preconditions	tests/fdcheck.py lists open descriptors >= 3
Steps / input	1. run 300 x true \ true; 2. python3 fdcheck.py
Expected result	output is []
Result of execution on 2026-10-03	PASS (as expected)

SC-12 - Parser fuzzing under sanitizers (security, SR-06)

Field	Content
Requirement(s)	SR-06
Preconditions	AddressSanitizer + UBSan build (make fuzz)
Steps / input	1. run build-asan/fuzz_parser (200,000 pseudo-random lines from a vocabulary of metacharacters, quotes, expansions and raw bytes)
Expected result	0 invariant violations and 0 sanitizer reports; program exits 0
Result of execution on 2026-10-03	PASS (as expected)

NF-01 - Responsiveness of background start (performance, NFR-01)

Field	Content
Requirement(s)	NFR-01
Preconditions	none
Steps / input	1. time mysh -c 'sleep 2 & echo started'
Expected result	elapsed time <= 500 ms
Result of execution on 2026-10-03	PASS (as expected)

NF-04 - Diagnostic format (usability, NFR-03)

Field	Content
Requirement(s)	NFR-03
Preconditions	12 error scenarios prepared
Steps / input	1. run each scenario; 2. check every stderr line
Expected result	100% of lines start with mysh: followed by a reason
Result of execution on 2026-10-03	PASS (as expected)

NF-07 - Recovery after errors (reliability, NFR-09)

Field	Content
Requirement(s)	NFR-09
Preconditions	none
Steps / input	1. pipe 10 faulty lines then echo alive into mysh
Expected result	stdout is alive only (no error terminated the shell)
Result of execution on 2026-10-03	PASS (as expected)

Appendix B - Test Case Catalogue and Results

All 130 tests below were executed on 2026-10-03. Results are copied from the result files by the document generator. <testdir> is the temporary directory created for each run.

B.1 Unit test cases

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
UT-01	FR-03	echo hello world	argc=3, argv[0]=echo, argv[3]=NULL	As expected	PASS
UT-02	FR-03	echo 'a b\ c;d'	2 args; second is a b\ c;d	As expected	PASS
UT-03	FR-13	echo "x \$UT_VAR y" (UT_VAR=value)	second arg: x value y	As expected	PASS
UT-04	FR-13	echo '\$UT_VAR'	second arg: \$UT_VAR (no expansion)	As expected	PASS
UT-05	FR-03	echo a\ b \> c	args: echo, 'a b', '>', c; no redirection	As expected	PASS
UT-06	FR-03	echo hi # this is ignored	argc=2	As	PASS

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
				expected	
UT-07	FR-18	cat f \ grep x \ wc -l	ncmds=3	As expected	PASS
UT-08	FR-14	sort < in.txt > out.txt	argc=1, infile=in.txt, outfile=out.txt, append=0	As expected	PASS
UT-09	FR-15	echo x>>log.txt	outfile=log.txt, append=1	As expected	PASS
UT-10	FR-20	sleep 5 &	background=1, argc=2	As expected	PASS
UT-11	FR-19	echo one; echo two	PARSE_OK 'one', PARSE_OK 'two', then PARSE_END	As expected	PASS
UT-12	FR-04	echo "unterminated	PARSE_ERROR, message mentions 'unterminated'	As expected	PASS
UT-13	FR-04	echo 'unterminated	PARSE_ERROR, 'unterminated single quote'	As expected	PASS
UT-14	FR-04	cat <	PARSE_ERROR, 'missing file name'	As expected	PASS
UT-15	FR-04	ls \ \ wc ; \ ls ; ls \	PARSE_ERROR for each	As expected	PASS
UT-16	FR-04	echo a && echo b ; echo a \ \ echo b	PARSE_ERROR mentioning && / \	As expected	PASS
UT-17	FR-13	echo \$? with last_status=42	second arg: 42	As expected	PASS
UT-18	FR-01	empty, blanks, '# only a comment', newline	PARSE_END for each	As expected	PASS
UT-19	FR-12	is_valid_name / is_assignment on PATH, _x9, 9x, A=1, =1, a-b=1	1,1,0,1,0,0 as appropriate	As expected	PASS
UT-20	FR-13	echo ~ ~/docs a~b (HOME=/home/unit)	/home/unit, /home/unit/docs, a~b	As expected	PASS
UT-21	FR-13	echo \${UT_BRACE}cd \$UT_UNSET end "\$UT_UNSET"	abcd, end, '' (empty kept because quoted)	As expected	PASS
UT-22	FR-04	echo \${bad	PARSE_ERROR 'bad substitution'	As expected	PASS
UT-23	FR-04	echo hi > ; > file	PARSE_ERROR for each	As expected	PASS
UT-24	FR-14	echo a >x >y	outfile=y	As expected	PASS

B.2 Functional test cases

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
TC-01	FR-09	pwd		As expected	PASS
TC-02	FR-05	ls file.txt input.txt	file.txt.txt	As expected	PASS
TC-03	FR-05	echo Hello	Hello	As expected	PASS
TC-04	FR-08	cd sub; pwd; cd ..; pwd	/sub	As expected	PASS
TC-06	FR-08	export HOME=<testdir>/sub; cd; pwd	/sub	As expected	PASS
TC-07	FR-08	cd sub; cd -		As expected	PASS
TC-10	FR-05	cat file.txt	hello worldagain	As expected	PASS
TC-11	FR-16	cat < input.txt	from input	As expected	PASS
TC-12	FR-14	echo hello > output.txt; cat output.txt	hello	As expected	PASS
TC-13	FR-14	echo one > t.txt; echo two > t.txt; cat t.txt	two	As expected	PASS
TC-14	FR-15	echo one > a.txt; echo two >> a.txt; cat a.txt	one	As expected	PASS
TC-15	FR-15	echo x >> new_app.txt; cat new_app.txt	x	As expected	PASS
TC-23	FR-12	export FOO=bar; echo \$FOO	bar	As expected	PASS
TC-26	FR-12	export A=1; unset A; echo "\$A"	[]	As expected	PASS
TC-28	FR-13	X=ab; echo \${X}cd	abcd	As expected	PASS
TC-29	FR-13	echo a \$NOPE_VAR b	a b	As expected	PASS
TC-30	FR-06	false; echo \$?; true; echo \$?	1	As expected	PASS
TC-31	FR-10	exit 7	[exit status 7]	As expected	PASS
TC-32	FR-10	echo a; exit 3; echo b	a [exit status 3]	As expected	PASS
TC-34	FR-11	help	usage lines for cd, pwd, exit, help, export, unset,	As expected	PASS

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
			env plus operator summary; status 0		
TC-37	FR-03	echo "a b"	a b	As expected	PASS
TC-38	FR-13	X=w; echo "hi \$X"	hi w	As expected	PASS
TC-39	FR-03	echo a\ b	a b	As expected	PASS
TC-40	FR-03	echo hi # comment	hi	As expected	PASS
TC-47	FR-01		blank / whitespace-only input does nothing	As expected	PASS
TC-56	FR-06	sh -c "kill -9 \\$\$"; echo \$?	137 [stderr contains: Killed]	As expected	PASS
TC-60	FR-13	export HOME=/tmp; echo ~ ~/x	/tmp /tmp/x	As expected	PASS
TC-61	FR-13	echo \$\$	a numeric process id	As expected	PASS
TC-71	FR-14	echo compact>c.txt; cat c.txt	compact	As expected	PASS
TC-72	FR-13	echo '\$HOME'	\$HOME	As expected	PASS
TC-76	FR-02	mysh --version	line beginning with "mysh"	As expected	PASS

B.3 Integration test cases

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
TC-18	FR-18	cat file.txt \ grep hello	hello worldagain	As expected	PASS
TC-19	FR-18	printf "b\na\nb\n" \ sort \ uniq	a	As expected	PASS
TC-20	FR-18	sort < file.txt \ head -n 1 > first.txt; cat first.txt	foo	As expected	PASS
TC-21	FR-18	pwd \ cat		As expected	PASS
TC-22	FR-14	pwd > pwd.txt; cat pwd.txt		As expected	PASS
TC-24	FR-12	export FOO=bar; sh -c "echo \ \$FOO"	bar	As expected	PASS
TC-25	FR-12	GREET=hi; echo \$GREET	hi	As	PASS

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
				expected	
TC-27	FR-12	export ZZ_TEST=1; env \\\ grep ZZ_TEST	ZZ_TEST=1	As expected	PASS
TC-35	FR-19	echo a; echo b; echo c	a	As expected	PASS
TC-36	FR-20	sleep 2 & echo started	prints “started” in < 1.5 s (does not wait 2 s)	As expected	PASS
TC-48	FR-05	export PATH=<testdir>/bin:\$PATH; hello_tool	tool-ran	As expected	PASS
TC-49	FR-05	./bin/hello_tool	tool-ran	As expected	PASS
TC-50	FR-02	mysh script.mysh (script: echo line1 / echo line2)	line1, line2	As expected	PASS
TC-51	FR-02	printf 'echo a\\necho b\\n' \\\ mysh	a, b and no prompt text	As expected	PASS
TC-52	FR-22	printf 'false\\n' \\\ mysh; echo \$?	exit status 1	As expected	PASS
TC-53	FR-10	mysh exit.mysh (script: echo first / exit 5 / echo never)	first; exit status 5; “never” not printed	As expected	PASS
TC-74	FR-20	sleep 1 \\\ sleep 1 & echo bg-pipeline	prints “bg-pipeline” in < 0.8 s	As expected	PASS
TC-75	FR-20	echo bgdata > bg.txt & sleep 0.5; cat bg.txt	bgdata	As expected	PASS

B.4 Error / negative test cases

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
TC-05	FR-08	cd nosuchdir; echo \$?	1 [stderr contains: No such file or directory]	As expected	PASS
TC-08	FR-07	invalidcommand; echo \$?	127 [stderr contains: command not found]	As expected	PASS
TC-09	FR-07	./noexec.sh; echo \$?	126 [stderr contains: Permission denied]	As expected	PASS
TC-16	FR-17	cat < missing.txt; echo \$?	1 [stderr contains: missing.txt: No such file]	As expected	PASS
TC-17	FR-17	echo hi > nodir/x.txt; echo \$?	1 [stderr contains: No such file or directory]	As expected	PASS

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
TC-33	FR-10	exit abc	[exit status 2] [stderr contains: numeric argument required]	As expected	PASS
TC-41	FR-04	echo "abc	[exit status 2] [stderr contains: unterminated double quote]	As expected	PASS
TC-42	FR-04	\ ls	[exit status 2] [stderr contains: syntax error]	As expected	PASS
TC-43	FR-04	ls \	[exit status 2] [stderr contains: syntax error]	As expected	PASS
TC-44	FR-04	cat <	[exit status 2] [stderr contains: missing file name]	As expected	PASS
TC-45	FR-04	echo a && echo b	[exit status 2] [stderr contains: '&&' is not supported]	As expected	PASS
TC-46	FR-04	echo before; echo "oops; echo after	before [exit status 2] [stderr contains: unterminated]	As expected	PASS
TC-62	FR-02	mysh -c	usage message; exit status 2	As expected	PASS
TC-63	FR-02	mysh /no/such/script	error message; exit status 127	As expected	PASS
TC-64	FR-12	export 1x=3; echo \$?	1 [stderr contains: not a valid identifier]	As expected	PASS
TC-65	FR-08	cd a b; echo \$?	1 [stderr contains: too many arguments]	As expected	PASS
TC-66	FR-07	./nothere; echo \$?	127 [stderr contains: No such file or directory]	As expected	PASS
TC-70	FR-04	> lonely.txt	[exit status 2] [stderr contains: redirection without a command]	As expected	PASS
TC-73	FR-04	false \ \ echo b	[exit status 2] [stderr contains: ' ' is not supported]	As expected	PASS

B.5 Robustness test cases

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
TC-54	NFR-02	echo 1 2 3 ... 2000 (2000 arguments)	exactly 2000 words printed	As expected	PASS

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
TC-55	NFR-02	echo <100000 x "a">	100001 bytes printed	As expected	PASS
TC-57	FR-18	echo x \ cat \ cat \ cat \ cat \ cat \ cat \ cat	x	As expected	PASS
TC-58	FR-18	nosuchcmd \ cat; echo \$?	0 [stderr contains: command not found]	As expected	PASS
TC-77	NFR-02	true \ true; (repeated 300 times) echo survived	survived	As expected	PASS

B.6 Security test cases

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
SC-01	SR-01	echo "a; echo injected \ cat > pwned"; ls pwned; echo ls-status=\$?	a; echo injected cat > pwned-status=2 [stderr contains: No such file]	As expected	PASS
SC-02	SR-02	X="a; echo pwned"; echo \$X	a; echo pwned	As expected	PASS
SC-03	SR-01	echo hi > 'weird;name\x'	exit status 0; one file named exactly weird;name x containing hi	As expected	PASS
SC-04	SR-01	grep -E '(system\ popen)s*\'('src/*.c and grep for exec of /bin/sh	no matches (user text is never handed to another shell)	As expected	PASS
SC-05	SR-03	grep for setuid/setgid/seteuid/setreuid in src/; mysh -c 'id -u'	no matches; child uid equals the invoking user's uid	As expected	PASS
SC-06	SR-04	session of 6 commands (pipe, cd, export, error, redirect to /dev/null) with HOME and cwd empty	no file created in HOME or cwd (no history, no log)	As expected	PASS
SC-07	SR-05	echo x > f under umask 077 / 022 / 000	file modes 600 / 644 / 644 (never more permissive than 0644)	As expected	PASS

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
SC-08	SR-07	export "a-b=1"; echo \$?; env \\ grep -c a-b	1 [exit status 1] [stderr contains: not a valid identifier]	As expected	PASS
SC-09	SR-08	300 x 'true \\\ true' then python3 fdcheck.py (lists inherited descriptors >= 3)	[]	As expected	PASS
SC-10	SR-08	mysh fd.mysh (script runs python3 fdcheck.py)	[] (the script file descriptor is close-on-exec)	As expected	PASS
SC-11	SR-10	printf 'SECRET' \\\ mysh -c 'cat & sleep 0.5'	no output: the background cat read /dev/null, not the shell's stdin	As expected	PASS
SC-12	SR-06	make fuzz -> build- asan/fuzz_parser (200,000 pseudo-random lines, ASan+UBSan)	0 invariant violations, 0 sanitizer reports, exit 0	As expected	PASS
SC-13	SR-06	make fuzz -> MYSH=build- asan/mysh python3 tests/fuzz_shell.py (600 random command lines)	0 crashes, 0 hangs, 0 sanitizer reports	As expected	PASS

B.7 Non-functional test cases

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
NF-01	NFR-01	sleep 2 & echo started	control returns in <= 500 ms although the job runs 2 s	As expected	PASS
NF-02	NFR-01	1000 x 'true' (fork+exec) in one -c string	total <= 5000 ms (mean <= 5 ms per external command)	As expected	PASS
NF-03	NFR-01	1000 x 'pwd > /dev/null' (built-in, no process)	total <= 1000 ms	As expected	PASS
NF-04	NFR-03	12 different error scenarios (bad command, bad cd, bad redirect, syntax errors, bad export/exit)	every stderr line starts with 'mysh:' followed by a reason	As expected	PASS
NF-05	NFR-04	gcc -std=c11 -Wall -Wextra -Wpedantic -Werror - fsyntax-only src/*.c	no warnings, no errors	As expected	PASS

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
NF-06	NFR-05	gcc -std=c99 -fsyntax-only src/*.c (C99 plus POSIX.1-2008 only)	compiles; no compiler-specific extensions needed beyond POSIX	As expected	PASS
NF-07	NFR-09	10 faulty lines (bad command, cd, redirect, syntax, export, permissions, && ...) then 'echo alive'	only 'alive' is printed: the shell survived every error	As expected	PASS
NF-08	NFR-03	help	at most 25 lines, no line longer than 80 characters, lists all 7 built-ins	As expected	PASS

B.8 System test cases

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
ST-01	FR-01	Start mysh on a pseudo-terminal	Prompt mysh:\$ appears	As expected	PASS
ST-02	FR-01,FR-05	echo hello-from-shell	Output shown, prompt returns	As expected	PASS
ST-03	FR-21,SR-09	Type 'partial text', press Ctrl+C, then echo still-alive	New prompt; shell alive; still-alive printed	As expected	PASS
ST-04	FR-21,FR-06,SR-09	sleep 30, wait 0.6 s, Ctrl+C, then echo \$?	Prompt within 5 s; \$? = 130	As expected	PASS
ST-05	FR-20	sleep 1 & ... wait 1.5 s ... press Enter	[1] at start; '[1]+ Done ... sleep 1' later	As expected	PASS
ST-06	FR-22	Ctrl+D at the prompt	'exit' printed; shell exits with status 0	As expected	PASS
ST-07	FR-21,FR-20	Start a background job (sh -c 'sleep 2; echo finished > f' &), press Ctrl+C at the prompt, wait	Job completes (file f exists) and Done is reported	As expected	PASS
ST-08	FR-21,SR-09	sleep 1, press Ctrl+Z	Job is not stopped; prompt returns; no hang	As expected	PASS
ST-09	FR-10	exit 4	Shell process exits with status 4	As expected	PASS
ST-10	FR-01..FR-22	mysh demo/demo.mysh	all documented demo outputs present; exit status	As expected	PASS

Test Case ID	Requirement ID	Input / Command	Expected Output	Actual Result	Pass/Fail
			0		
ST-11	FR-21,FR-20	Background job, then sleep 30 in foreground, Ctrl+C	Foreground killed; background job still completes (file exists)	As expected	PASS
ST-12	NFR-02,NFR-08,SR-06	make sanitize	0 sanitizer reports; unit and functional suites all pass	As expected	PASS

B.9 Tests still to be executed (not run, no result claimed)

- Regression run after review comments and on each team member's own machine (planned 05-Oct-2026), including a run as an ordinary (non-root) user.
- Build and test on a non-Linux POSIX system.
- Pseudo-terminal suite under the sanitizer build.
- Coverage-guided fuzzing and third-party penetration test. Acceptance demonstration to the instructor.